Operating Systems

UE23CS242B

4th Semester, Academic Year 2023

Date: 17/03/2025

Name: Roshini Ramesh	SRN: PES1UG23CS488	Section: H

QUESTION 3:

SIMULATE A TWO-LEVEL PAGING SYSTEM TO CONVERT A LOGICAL ADDRESS (OUTER PAGE, INNER PAGE, OFFSET) TO A PHYSICAL ADDRESS USING NESTED PAGE TABLES. (MAINTAIN AN OUTER PAGE TABLE THAT MAPS TO AN INNER PAGE TABLE). DISPLAY ERROR MESSAGES IN CASE OF FAULTS.

CODE:

```
#include <stdio.h>

#include <stdlib.h>


#define OUTER_PAGES 4

#define INNER_PAGES 4
```

```
#define PAGE_SIZE 256

int *outer_page_table[OUTER_PAGES];

void initialize_page_tables() {
   for (int i = 0; i < OUTER_PAGES; i++) {
      outer_page_table[i] = NULL;
   }

   outer_page_table[1] = (int *)malloc(INNER_PAGES * sizeof(int));
   outer_page_table[2] = (int *)malloc(INNER_PAGES * sizeof(int));

   for (int i = 0; i < INNER_PAGES; i++) {
      if (outer_page_table[1]) outer_page_table[1][i] = 10 + i; // Frames 10, 11, 12, 13
      if (outer_page_table[2]) outer_page_table[2][i] = 20 + i; // Frames 20, 21, 22, 23
   }
}
```

```
void logical_to_physical(int outer_page, int inner_page, int offset) {
   if (outer_page >= OUTER_PAGES || inner_page >= INNER_PAGES ||
offset >= PAGE_SIZE) {
      printf("Error: Invalid address\n");
      return;
   }

   if (outer_page_table[outer_page] == NULL) {
      printf("Page Fault: Outer page %d not mapped\n", outer_page);
      return;
   }

   int frame_number = outer_page_table[outer_page][inner_page];

   if (frame_number == -1) {
      printf("Page Fault: Inner page %d not mapped\n", inner_page);
      return;
   }

   int physical_address = (frame_number * PAGE_SIZE) + offset;
```

```
    printf("Logical Address (%d, %d, %d) → Physical Address: %d\n",
           outer_page, inner_page, offset, physical_address);
}

void allot_page(int outer_page, int inner_page, int frame_number) {
    if (outer_page >= OUTER_PAGES || inner_page >= INNER_PAGES) {
        printf("Error: Invalid page allocation request.\n");
        return;
    }

    if (outer_page_table[outer_page] == NULL) {
        outer_page_table[outer_page] = (int *)malloc(INNER_PAGES *
sizeof(int));
        for (int i = 0; i < INNER_PAGES; i++) {
            outer_page_table[outer_page][i] = -1;
        }
    }

    outer_page_table[outer_page][inner_page] = frame_number;
}
```

```
void clean_tables() {
    for (int i = 0; i < OUTER_PAGES; i++) {
        if (outer_page_table[i] != NULL) {
            free(outer_page_table[i]);
            outer_page_table[i] = NULL;
        }
    }
}

int main() {
    initialize_page_tables();

    printf("Test 1:\n");
    logical_to_physical(1, 2, 128);
    printf("\nTest 2:\n");
    logical_to_physical(2, 1, 100);
    printf("\nTest 3:\n");
    logical_to_physical(3, 0, 50);
```

```
    printf("\nTest 4:\n");
    logical_to_physical(1, 5, 10);
    printf("\nTest 5:\n");
    logical_to_physical(0, 1, 200);
    printf("\nTest 6:\n");
    allot_page(0, 1, 30);
    logical_to_physical(0, 1, 100);

    clean_tables();
    return 0;
}
```

OUTPUT:

